



Ecole Universitaire de Physique et d'Ingenierie

3D Deep Learning

2025–2026

Rapport de projet

Classification de nuages de points 3D avec PointNet

Master PAR — Lab PointNet

Realise par
NOCHI MAGOUC
Paul CHECCHIN

Encadrants
Jean-Emmanuel DESCHAUD

24 février 2026

Table des matières

Resume	1
1 Introduction	2
1.1 Contexte	2
1.2 Objectifs du projet	2
1.3 Perimetre experimental	2
1.4 Contributions du rapport	2
2 Methodes	3
2.1 Donnees et pretraitement	3
2.2 PointMLP (Exercice 1)	3
2.3 PointNetBasic (Exercice 2.1)	3
2.4 PointNetFull avec T-Net (Exercice 2.2)	3
2.5 Augmentations 3D comparees (Exercice 3)	3
3 Protocole experimental	4
3.1 Jeux de donnees et splits	4
3.2 Configuration du notebook final	4
3.3 Campagnes automatisees complementaires	4
3.4 Metriques reportees	4
4 Resultats	6
4.1 Campagne augmentation seed 42 (25 epochs)	6
4.2 Benchmark robuste (seed 1, max 50 epochs)	6
4.3 Run de reference du notebook final	7
4.4 Run cible occlusion (seed 0)	7
5 Discussion	9
5.1 Lectures principales	9
5.2 Interpretation	9
5.3 Limites actuelles	9
5.4 Etat en cours	9
6 Conclusion	10
Bibliographie	11

Table des figures

4.1	Comparaison globale des <code>best_val_acc</code> (campagne du 08/02/2026).	6
4.2	Courbes d'entraînement du run <code>plus_occlusion, seed_0</code>	7
4.3	Comparaison qualitative des matrices de confusion.	8

Liste des tableaux

3.1	Campagnes experimentales utilisees.	4
4.1	Top configurations — campagne 20260208_145104.	6
4.2	Synthese — benchmark robuste 20260211_081948.	7

Resume

Ce rapport presente l'implementation en PyTorch des exercices PointMLP, PointNetBasic et PointNetFull (T-Net 3×3), puis l'etude de plusieurs strategies de data augmentation sur ModelNet10. La campagne de reference du notebook final atteint **90.09%** d'accuracy test avec la configuration *baseline + occlusion*. Les campagnes automatisees confirment que l'occlusion reste l'une des augmentations les plus robustes selon les metriques de validation/evaluation disponibles.

Chapitre 1

Introduction

1.1 Contexte

Les nuages de points 3D sont une representation compacte de la geometrie, mais ils sont non structures et non ordonnes. Les architectures CNN classiques sur grilles regulares sont donc mal adaptees sans conversion (voxels, projections multi-vues). PointNet traite directement les points en combinant des MLP partages et une aggregation symetrique de type max-pooling [1].

1.2 Objectifs du projet

L'enonce du *PointNet Lab* fixe trois objectifs principaux :

- implementer un **PointMLP** (Exercice 1) et mesurer sa performance de classification ;
- implementer **PointNetBasic** puis **PointNetFull avec T-Net 3×3** (Exercice 2), et comparer les resultats ;
- proposer et evaluer une nouvelle augmentation 3D (Exercice 3), en comparaison avec la baseline.

1.3 Perimetre experimental

Les experiments sont menes sur **ModelNet10_PLY** (1024 points par objet, 10 classes), issu de la famille ModelNet [2]. Le code final est consolide dans `Code/pointnet_notebook.ipynb`, avec une logique de split train/validation/test et des campagnes automatisees de benchmark dans `Code/reports`.

1.4 Contributions du rapport

Ce document apporte :

- une synthese claire des choix d'architecture et d'entrainement ;
- un protocole experimental reproductible (seeds, early stopping, metriques) ;
- une lecture critique des campagnes d'augmentation, avec focus sur *baseline + occlusion* ;
- un etat des lieux coherent avec les sorties deja generees et le fine-tuning en cours (seed 45).

Chapitre 2

Methodes

2.1 Donnees et pretraitement

Chaque objet est represente par un nuage de 1024 points (x, y, z) . Le pipeline de base applique la normalisation, la rotation autour de z , un bruit gaussien et le melange des points. Pour les tests finaux, la transformation est volontairement plus stable (normalisation uniquement).

2.2 PointMLP (Exercice 1)

PointMLP applique un flatten $1024 \times 3 = 3072$ puis un MLP global :

$$3072 \rightarrow 512 \rightarrow 256 \rightarrow N_{classes} \quad (2.1)$$

avec BatchNorm, ReLU, Dropout ($p = 0.3$) et LogSoftmax en sortie. Cette architecture sert de baseline simple, sans invariance explicite a l'ordre des points.

2.3 PointNetBasic (Exercice 2.1)

PointNetBasic utilise des MLP partages (Conv1d noyau 1) sur chaque point, puis un max-pooling global. Le descripteur global est ensuite classe par un MLP fully-connected. Cette structure rend le modele invariant a la permutation des points.

2.4 PointNetFull avec T-Net (Exercice 2.2)

La version complete ajoute un T-Net 3×3 qui apprend une transformation d'alignement en entree. La perte totale est :

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \left\| I - AA^T \right\|_F^2 \quad (2.2)$$

ou A est la matrice predite par le T-Net. Le second terme regularise la transformation vers une matrice proche d'une rotation.

2.5 Augmentations 3D comparees (Exercice 3)

Les campagnes automatisees comparent plusieurs variantes autour d'une baseline commune :

- **baseline** : rotation_z + bruit gaussien ;
- **augmentations unitaires** : jitter, mirror, scale, translate, dropout, occlusion ;
- **combinaisons** : combo_geo, combo_robust, combo_full.

L'augmentation la plus suivie dans la phase finale est l'**occlusion spherique aleatoire** (rayon et probabilite controles), car elle force le modele a apprendre des indices partiels plus robustes.

Chapitre 3

Protocole experimental

3.1 Jeux de donnees et splits

Le notebook final utilise :

- train : **3193** exemples,
- validation : **798** exemples,
- test : **908** exemples.

Le split est derive du train officiel avec `VAL_RATIO = 0.2` et `seed = 42`, puis le test officiel est conserve pour l’évaluation finale.

3.2 Configuration du notebook final

Configuration extraite de `Code/pointnet_notebook.ipynb` :

- modele : `PointNetFull`,
- augmentations train : `[normalize, rot_z, noise, shuffle, occlusion]`,
- augmentation test : `[normalize]`,
- `MAX_EPOCHS = 150`,
- early stopping active (monitor `val_acc`, patience 50, min_delta 0.0),
- scheduler de taux d’apprentissage actif.

3.3 Campagnes automatisees complementaires

En parallele du notebook, des campagnes de benchmark ont ete lancees avec le script robuste. Table 3.1 resume les runs principaux exploites dans ce rapport.

TABLE 3.1 – Campagnes experimentales utilisees.

Campagne	Date	Usage dans le rapport
C1 (20260208_145104)	08/02/2026	benchmark de 10 augmentations (seed 42, 25 epochs)
C2 (20260211_081948)	11/02/2026	benchmark robuste (4 experiences, seed 1, max 50 epochs, early stop)
C3 (20260211_143359)	11/02/2026	run cible <code>plus_occlusion</code> , <code>seed_0</code> (max 150 epochs, monitor <code>val_loss</code>)

3.4 Metriques reportees

Selon les sorties disponibles, on suit principalement :

- `best_val_acc` et `best_val_loss`,
- `best_eval_acc` quand presente,
- `final_test_acc` pour le notebook final,
- matrices de confusion et courbes d'apprentissage.

Cette distinction est importante pour eviter de melanger validation interne et evaluation finale.

Chapitre 4

Resultats

4.1 Campagne augmentation seed 42 (25 epochs)

Le benchmark du 8 fevrier 2026 compare 10 configurations autour de PointNetFull. Les meilleures configurations en **best_val_acc** sont presentees dans le tableau 4.1.

TABLE 4.1 – Top configurations — campagne 20260208_145104.

Configuration	best_val_acc (%)	best_epoch	final_val_acc (%)
plus_jitter	91.08	23	86.67
plus_mirror	90.75	21	87.56
plus_occlusion	90.42	23	89.76
baseline	90.31	23	88.77
combo_full	90.09	22	90.09

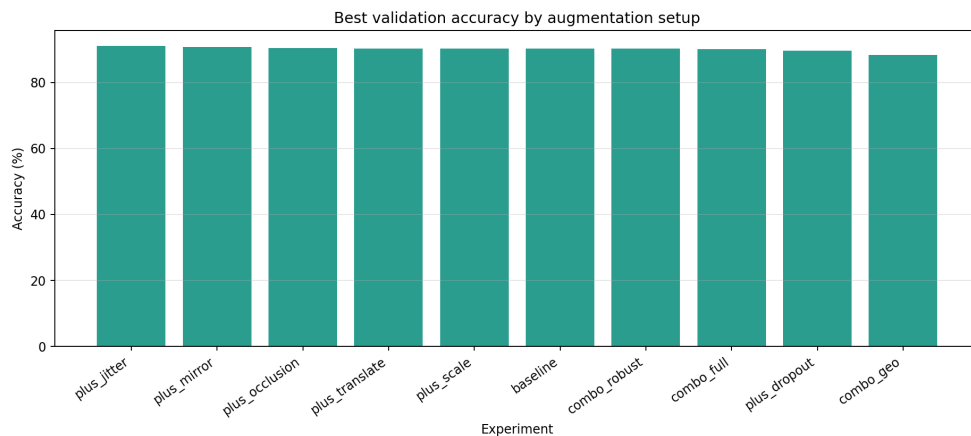


FIGURE 4.1 – Comparaison globale des **best_val_acc** (campagne du 08/02/2026).

4.2 Benchmark robuste (seed 1, max 50 epochs)

Le benchmark du 11 fevrier 2026 confirme que **plus_occlusion** reste en tete parmi les 4 configurations testees (Table 4.2).

TABLE 4.2 – Synthèse — benchmark robuste 20260211_081948.

Configuration	best_val_acc (%)	best_epoch	final_val_acc (%)
plus_occlusion	88.33	21	87.33
combo_full	88.22	18	88.00
baseline	86.45	10	81.61
plus_jitter	86.12	7	83.81

4.3 Run de reference du notebook final

Le run final du notebook (seed 42) donne :

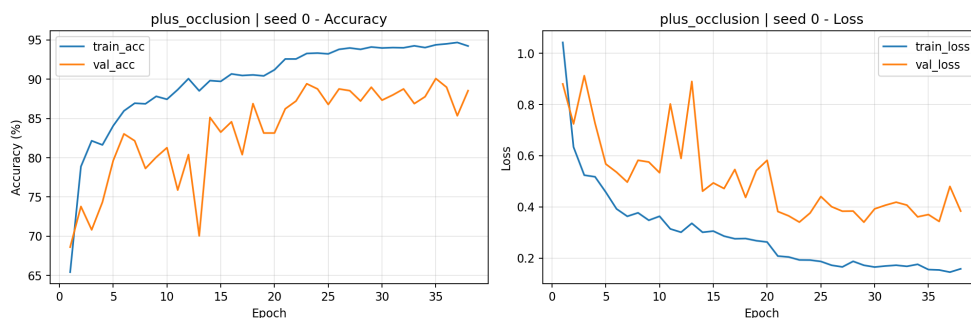
- MAX_VAL_ACC = 94.36% (sur 150 epochs),
- Final TEST accuracy = 90.09%,
- Final TEST loss = 0.4077.

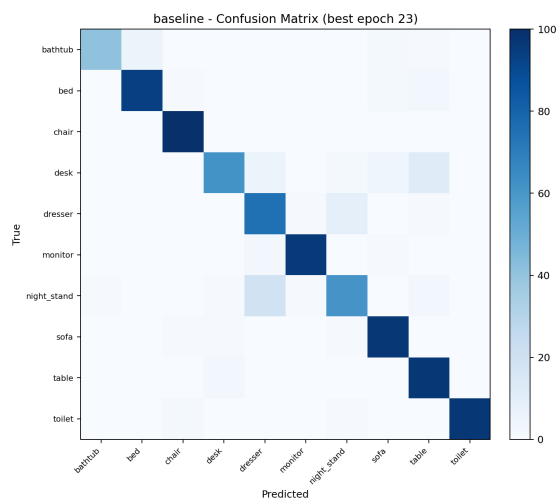
Ces valeurs sont celles affichees directement dans la sortie du notebook final.

4.4 Run cible occlusion (seed 0)

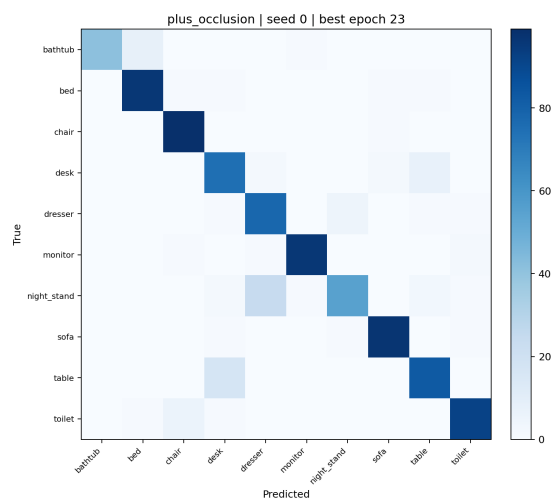
Dans la campagne 20260211_143359, le run `plus_occlusion/seed_0` atteint :

- best_val_acc = 90.09% (epoch 23),
- best_eval_acc = 89.43%,
- final_val_acc = 88.55% apres early stopping.

FIGURE 4.2 – Courbes d’entrainement du run `plus_occlusion, seed_0`.



(a) Baseline



(b) Plus occlusion (seed 0)

FIGURE 4.3 – Comparaison qualitative des matrices de confusion.

Chapitre 5

Discussion

5.1 Lectures principales

Trois constats ressortent des campagnes disponibles :

- l'**occlusion** est systématiquement compétitive et souvent première dans les protocoles les plus contraints ;
- les combinaisons lourdes d'augmentations ne garantissent pas de gain stable par rapport à une politique plus simple ;
- la configuration du monitor d'early stopping (**val_acc** vs **val_loss**) influence notablement l'époque retenue et les métriques finales.

5.2 Interpretation

L'occlusion aléatoire est pertinente pour ModelNet10 car elle force le réseau à reconnaître des formes partielles. Cela améliore la robustesse à la disparition locale de points, ce qui est cohérent avec des scènes réelles où les acquisitions sont incomplètes. Le gain en test du notebook final (90.09%) valide ce choix pour la version courante du projet.

5.3 Limites actuelles

- les benchmarks robustes ne couvrent pas encore un vrai multi-seeds complet (plusieurs tables ont **num_runs** = 1) ;
- la comparaison Exercice 1 / Exercice 2 reste surtout architecturale dans cette version du rapport, faute de tableaux consolidés homogènes sur la même grille d'essais ;
- l'évaluation est concentrée sur ModelNet10, sans extension ModelNet40.

5.4 Etat en cours

Au moment de rédaction, un fine-tuning **baseline** + **occlusion** en **seed** 45 est en cours. Il est prévu d'ajouter ses métriques finales (accuracy, loss, matrice de confusion) dans le tableau de comparaison principal dès que le run est terminé.

Chapitre 6

Conclusion

Ce projet a permis d'implémenter les trois briques demandées dans l'énoncé (PointMLP, PointNetBasic, PointNetFull avec T-Net 3×3) et de structurer un protocole expérimental reproductible.

Les campagnes montrent qu'une stratégie d'augmentation **simple et ciblée** (notamment l'occlusion) est plus robuste qu'une accumulation systématique de transformations. Le meilleur résultat actuellement documenté dans le notebook final atteint **90.09%** d'accuracy test sur ModelNet10.

La suite immédiate consiste à consolider la comparaison multi-seeds (dont seed 45 en cours), puis à figer une table finale *mean $\pm$ std* pour la version définitive du rapport.

Bibliographie

- [1] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet : Deep learning on point sets for 3d classification and segmentation. In *CVPR*, 2017.
- [2] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets : A deep representation for volumetric shapes. *CVPR*, 2015.